IT-TAGE 2026 · FRANKFURT · 10 DECEMBER 2026

Aspire in Practice

From local development to tests and deployment.

Sven Hennessen · vensas GmbH

vensas

About me



- **Almost 20 years** of software development and architecture
- **Co-founder and co-managing director** of vensas GmbH, founded in 2023
- Consulting and development of **business software** for medium-sized companies
- Away from the computer — **hiking**, **cycling** and **snowboarding**

Sven Hennessen
vensas GmbH

From Tye to Aspire

2019
Project Tye
A Microsoft experiment. A CLI that starts all services.

→

2023
.NET Aspire preview
Build 2023. A tool for the local developer.

→

2024
Aspire GA
May 2024, on .NET 8. A stable API.

→

2025
Aspire on its own
Not tied to .NET releases. Its own version numbers.

This talk uses Aspire 13.6 on .NET 10.

The demo app: the cookie jar

A sensor takes cookies from the office cookie jar. The page shows the fill level in real time.

Sensor	Python, sends cookie-taken events
Broker	RabbitMQ
Server	ASP.NET Core and EF Core, two replicas
Database	PostgreSQL
Frontend	React and Vite, Server-Sent Events



One AppHost for three languages

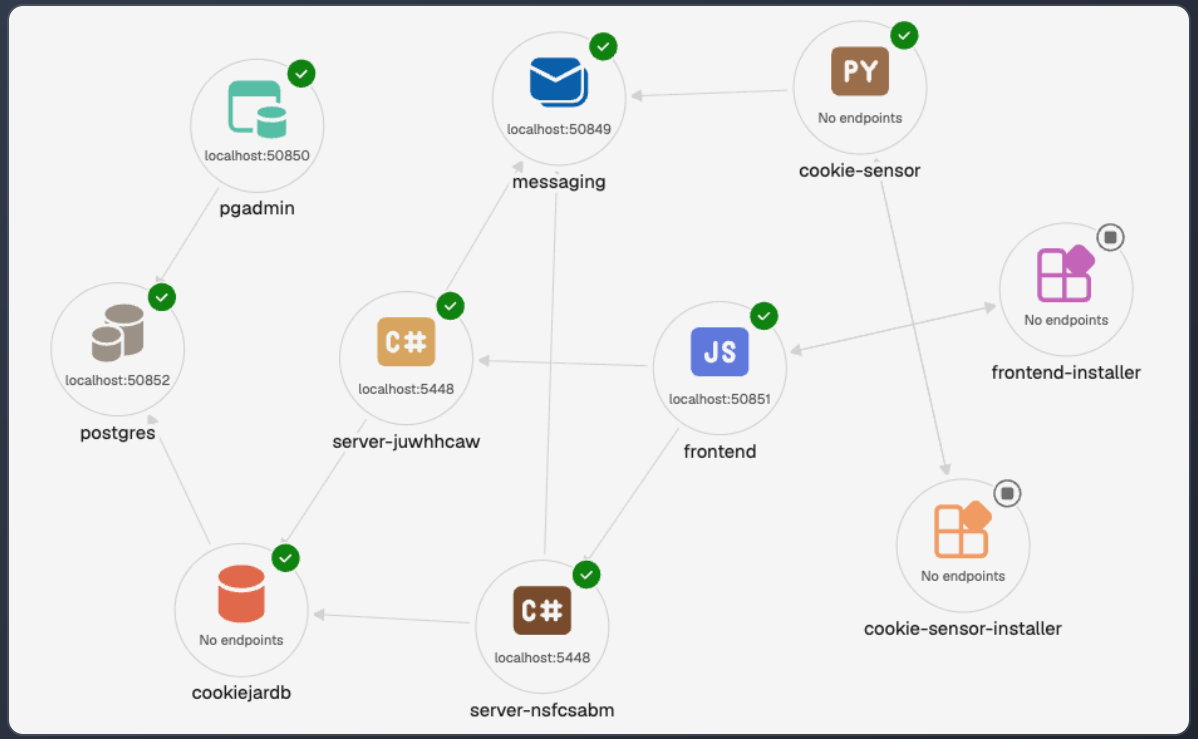
AppHost.cs

```
var server = builder
  .AddProject<Projects.CookieJar_Server>("server")
  .WithReference(db).WaitFor(db)
  .WithReference(rabbitmq).WaitFor(rabbitmq)
  .WithReplicas(2);

builder.AddPythonApp("cookie-sensor", ...)
  .WithReference(rabbitmq);

builder.AddViteApp("frontend", "../frontend")
  .WithReference(server);
```

aspire run starts all of it. WaitFor waits until the resource is healthy.

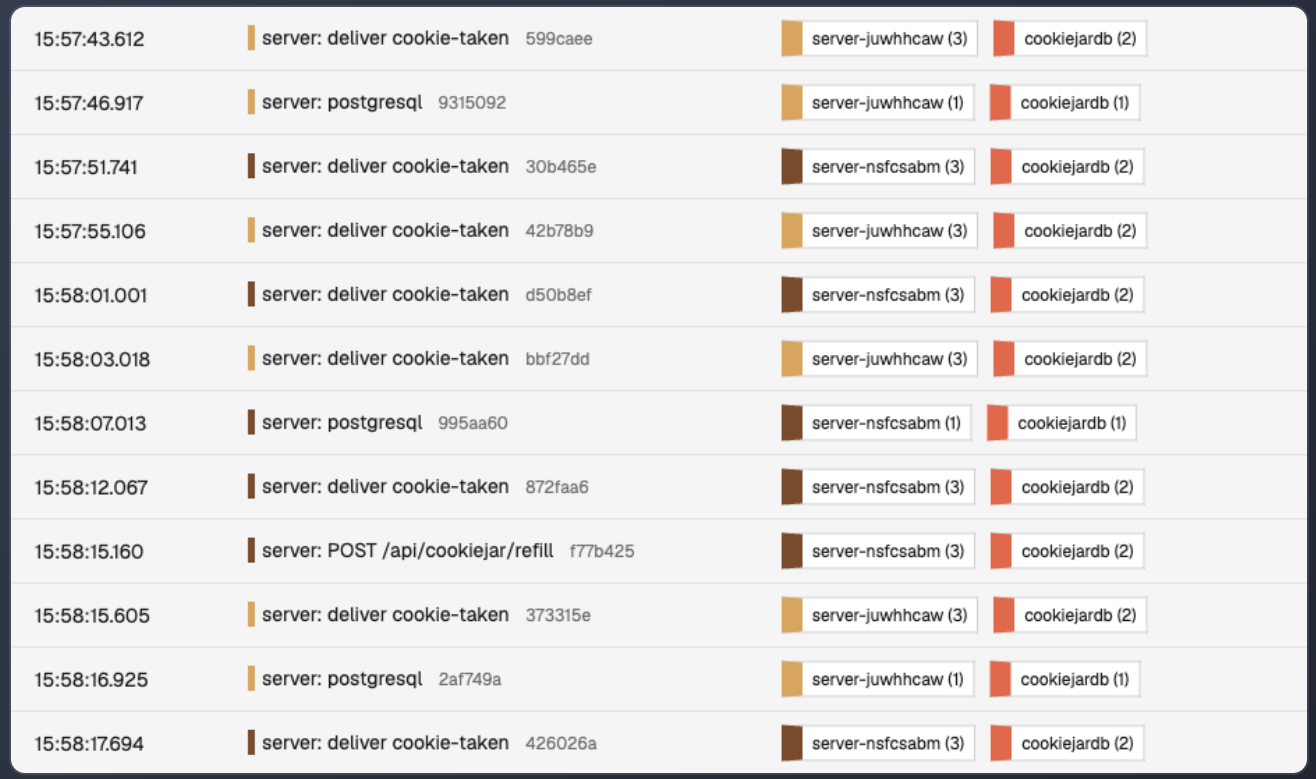


Logs, traces and metrics in one dashboard

What you get

- OpenTelemetry from AddServiceDefaults()
- Console and structured logs for each replica
- Traces across the services
- Start, stop and restart for each resource

The two replicas take turns. RabbitMQ gives each event to one of them.

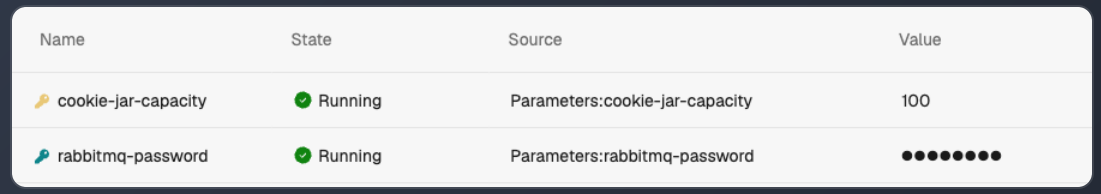


Parameters and secrets

AppHost.cs

```
// A secret has no default value
var password = builder.AddParameter(
    "rabbitmq-password", secret: true);

// Read the configuration, then use the default
var capacity = builder.AddParameter(
    "cookie-jar-capacity",
    config["Parameters:cookie-jar-capacity"] ?? "100");
```



Name	State	Source	Value
cookie-jar-capacity	Running	Parameters:cookie-jar-capacity	100
rabbitmq-password	Running	Parameters:rabbitmq-password	●●●●●●●●

Watch out

AddParameter(name, "100") sets a constant value. Aspire then ignores the configuration. Read the configuration first, as above.

How Aspire works



Model	Each resource is an IResource with annotations. Build() makes the model real.
WithReference	Gives ConnectionStrings__name as an environment variable, with the dynamic port.
WaitFor	Holds the start until the dependency is healthy.

Aspire.Hosting.Testing

The AppHost is the test fixture. The test gets the same topology as aspire run.

AppModelTests.cs

```
var builder = await DistributedApplicationTestingBuilder
    .CreateAsync<Projects.CookieJar_AppHost>();

var server = builder.Resources
    .Single(r => r.Name == "server");
Assert.Equal(2, server.GetReplicaCount());
```

Model test: no containers, less than 1 s.

CookieJarAppFixture.cs

```
App = await builder.BuildAsync(ct);
await App.StartAsync(ct);
await App.ResourceNotifications
    .WaitForResourceHealthyAsync("server", ct);

var http = App.CreateHttpClient("server");
```

E2E test: real containers, one start for all tests.

Change the AppHost for the test

Before BuildAsync() the model is only data. You can remove, replace and add resources.

CookieJarAppFixture.cs

```
// Parameters as arguments. The build agent needs no user secrets.
var builder = await DistributedApplicationTestingBuilder.CreateAsync<Projects.CookieJar_AppHost>(
    ["Parameters:rabbitmq-password=e2e-test", "Parameters:cookie-jar-capacity=42"]);

// No UI and no random sensor. The test sends the events itself.
foreach (var r in builder.Resources.Where(r => r is PgAdminContainerResource
             || r.Name.StartsWith("frontend") || r.Name.StartsWith("cookie-sensor")).ToList())
    builder.Resources.Remove(r);

// An empty database. Do not touch the volume of the local environment.
foreach (var mount in postgres.Annotations.OfType<ContainerMountAnnotation>().ToList())
    postgres.Annotations.Remove(mount);
```

Recommendation

Remove the data volume in the test. Each run then starts with an empty database.

Two replicas in the test

Parameter	The test gives capacity 42 as an argument. A refill must return 42.
Migration	Both replicas start at the same time. An advisory lock lets only one of them migrate.
SSE	Each replica streams only the events that it consumed. The explicit test shows this limit.

One instance hides these problems. With WithReplicas(2) the test shows them.

```
$ dotnet test src/CookieJar.Tests
Passed!  - Failed: 0, Passed: 6, Skipped: 0, Total: 6, Duration: 9 s

$ dotnet run --project src/CookieJar.Tests -- -explicit only
    CookieJarTests.LevelStream_OnEveryReplica_ReceivesEveryChange [FAIL]
      Assert.Equal() Failure: Collections differ
                              ↓ (pos 3)
      Expected: [100, 99, 98, 97, 96]
      Actual:   [100, 99, 98]
   CookieJar.Tests  Total: 7, Failed: 1, Not Run: 6, Time: 52.6s
```

FROM A REAL PROJECT

A delivery service with two replicas

E2ETestHost.cs

```
// Containers only for this run. No shared data.
foreach (var r in appHost.Resources.OfType<ContainerResource>())
    r.Annotations.Add(new ContainerLifetimeAnnotation
        { Lifetime = ContainerLifetime.Session });

// The upstream system is a fake in the test process
appHost.CreateResourceBuilder<ProjectResource>("api")
    .WithEnvironment("UPSTREAM_URL", fakeUpstream.Url);

// Chaos: freeze the storage, then check the behavior
await DockerAsync("pause", storageContainerId);
```

What the tests check

Lock	Only one replica migrates. The cron job runs one time per tick.
Outage	The cache answers. Else 503 with Retry-After.
Runtime	Nightly, about 8 minutes.

Recommendation

Run the E2E tests nightly. An environment variable turns the project on, thus the PR build skips it.

FROM A REAL PROJECT

A business application with a web UI

AspireAppFixture.cs

```
appHost.CreateResourceBuilder<ProjectResource>("api")
    // The real external system is not reachable
    .WithEnvironment("EXTERNAL__MASTERDATA__BASEADDRESS",
        "http://127.0.0.1:9")
    // Healthy = the API runs and asks for a login
    .WithHttpHealthCheck("/v1/me", statusCode: 401);

await TestData.SeedAsync(NpgsqlDataSource.Create(
    await app.GetConnectionStringAsync("app-db")));
```

How the team works

- One test for each acceptance criterion
- UI, API and database in one run
- Playwright keeps the trace only after a failure

Recommendation

Make each external system unreachable in the test. Then no test can call it by mistake.

Docker Compose

AppHost.cs

```
builder.AddDockerComposeEnvironment("docker-compose")
    .ConfigureComposeFile(compose =>
    {
        compose.Services["server"]
            .DependsOn["postgres"]
            .Condition = "service_healthy";
    });

// aspire publish --output-path ./deploy/output
```

Azure is a first-class target too. azd up reads the same model. It is not part of this demo.

deploy/output/docker-compose/docker-compose.yaml

```
server:
  image: "${SERVER_IMAGE}"
  environment:
    CookieJar__Capacity: "${COOKIE_JAR_CAPACITY}"
    MESSAGING_PASSWORD: "${RABBITMQ_PASSWORD}"
  depends_on:
    postgres:
      condition: "service_healthy"
```

Generated. The secrets come from .env.

Kubernetes and Kamal

AppHost.cs

```
builder.AddKubernetesEnvironment("kubernetes");

// aspire publish → deploy/output/helm
// helm upgrade --install cookiejar \
//     ./deploy/output/helm
```

Kubernetes (preview): a Helm chart. Use it as a start, not as the final chart.

AppHost.cs

```
builder.AddKamalEnvironment("kamal")
    .WithServers(vmIp)
    .WithRegistry(registry, "unused")
    .WithProxyHostSuffix($"{vmIp}.nip.io");

server.PublishAsKamalService((_, config) =>
    config.Proxy!.Healthcheck =
        new() { Path = "/health" });
```

Kamal: the open-source NuGet package Vensas.Aspire.Hosting.Kamal.

Kamal: from the app model to deploy.yml

AddProject, AddPythonApp	One app with its own deploy.name.yml
Containers	Accessories of the first app
WithExternalHttpEndpoints	proxy: host, app_port, health check
Secret parameters	.kamal/secrets, references only
WithReference(server)	Network alias server-web

```
dotnet add package Vensas.Aspire.Hosting.Kamal
```

deploy/output/kamal/config/deploy.yml

```yaml
service: server
image: cookiejar/server
proxy:
  host: server.192.168.64.5.nip.io
  app_port: 8080
  healthcheck:
    path: /health
env:
  secret:
    - ConnectionStrings__cookiejardb
accessories:
  postgres:
    image: docker.io/library/postgres:18.3
```

Generated by aspire publish.

Aspire MCP: the agent sees the running app

The MCP server gives the agent the resources, logs and traces of the running app. Not only the code.

```
aspire agent init     # writes the MCP configuration for Claude Code, Copilot and others
```

STEP 1
Prompt
Why does replica 2 not stream the new level?

→

STEP 2
Resources
list_resources gives the state of each replica.

→

STEP 3
Logs
list_structured_logs shows which replica consumed the event.

→

STEP 4
Answer
The cause, with log lines. You decide on the fix.

TL;DR

1. **Use Aspire for local development now.**
   The full stack with one command, in any language, with OpenTelemetry.
2. **Test with the real AppHost and two replicas.**
   Aspire.Hosting.Testing shows the problems that one instance hides.
3. **Outside Azure, plan time for the deployment.**
   Compose and Helm are a start. Kamal shows that the provider model works.
4. **AI: give the agent the running app with Aspire MCP.**
   Resources, logs and traces are the context. A human decides on the fix.

That's all folks!

THANK YOU

Questions?

Demo project and slides available at https://github.com/vensas/aspire-2026-talk

https://www.linkedin.com/in/svenhennessen/

kontakt@vensas.de

vensas.de

github.com/vensas/aspire-hosting-kamal



Slides & Demo